

IF5153 Natural Language Processing
Laporan Praktikum 3
Text Classification using Deep Learning

Disusun untuk memenuhi tugas mata kuliah Pemrosesan Bahasa Alami
pada Semester 1 (satu) Tahun Akademik 2025/2026
Tugas Besar IF5153 Pemrosesan Bahasa Alami



Oleh

Shabrina Maharani

13522134

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2025

I. Deskripsi Persoalan

Text classification adalah salah satu task dalam bidang Natural Language Processing (NLP) yang bertujuan untuk mengelompokkan teks ke dalam kategori tertentu berdasarkan isi atau maknanya. Salah satu penerapannya adalah analisis sentimen, yaitu mengidentifikasi apakah suatu teks mengandung opini positif, negatif, atau netral.

Pada praktikum kali ini, mahasiswa diminta untuk membangun sistem klasifikasi teks sentimen dengan menggunakan pendekatan deep learning. Tahapan yang dilakukan meliputi preprocessing data, feature extraction menggunakan metode tradisional seperti Bag of Words dan TF-IDF, serta metode berbasis word embeddings seperti Word2Vec, GloVe, dan FastText. Selain itu, dilakukan juga perbandingan dengan teknik SVD untuk reduksi dimensi.

Model klasifikasi yang digunakan adalah beberapa algoritma non-transfer deep learning, seperti CNN, RNN, dan LSTM. Tujuan akhirnya adalah membandingkan performa dari berbagai kombinasi metode ekstraksi fitur dan algoritma deep learning untuk menentukan pendekatan yang paling efektif dalam menghasilkan model analisis sentimen.

II. Penjelasan Preprocessing

Tahap preprocessing bertujuan untuk mengubah teks mentah menjadi bentuk yang lebih terstruktur sehingga bisa diolah oleh model deep learning. Dari hasil EDA, terlihat bahwa dataset sentimen cukup imbalanced (kelas negatif jauh lebih banyak dibandingkan kelas positif: 493 vs 93). Selain itu, distribusi panjang teks umumnya berkisar antara 40–120 kata, sehingga preprocessing juga penting untuk menyeragamkan bentuk teks sebelum masuk ke tahap feature extraction. Wordcloud dan analisis unigram/bigram menunjukkan kata-kata umum seperti life, new, friend, day, moment mendominasi, sehingga filtering kata umum dan normalisasi teks juga diperlukan agar model lebih fokus pada kata-kata yang membawa makna sentimen.

1. **Lowercasing** dengan semua teks diubah menjadi huruf kecil agar bentuk kata lebih seragam dan tidak terjadi duplikasi token karena perbedaan kapitalisasi.

2. **Cleaning** dengan menghapus HTML tag, URL, mention, angka, dan tanda baca, mengurangi huruf berulang (contoh: soooo → soo), dan mengubah emoji menjadi teks deskriptif (contoh: 😊 → :smile:).
3. **Handling contractions** dengan membuat bentuk singkatan seperti *don't* diubah menjadi *do not* supaya konsisten dengan tokenisasi.
4. **Tokenisasi dan Lemmatization** dengan cara teks dipecah menjadi token, lalu setiap kata dikembalikan ke bentuk dasar menggunakan **WordNetLemmatizer** (misalnya *running*, *ran* → *run*).
5. **Negation Handling** untuk menggabungkan kata negasi dengan kata sesudahnya (misalnya *not good* → *not_good*) untuk menjaga konteks semantik.
6. **Stopword removal dengan pengecualian** jadi stopwords umum dihapus, tetapi kata-kata penting yang sering berfungsi sebagai penentu sentimen seperti *not*, *but*, *very*, *however* tetap dipertahankan.
7. **Negation handling** untuk menjaga konteks semantik, kata negasi digabungkan dengan kata setelahnya, misalnya *not good* → *good_NEG*. Hal ini membantu model membedakan sentimen sebenarnya.

III. Feature Extraction

Setelah teks melalui tahap preprocessing, langkah berikutnya adalah mengubah teks menjadi representasi numerik agar bisa diproses oleh algoritma *machine learning*. Dalam eksperimen ini digunakan beberapa metode ekstraksi fitur dengan karakteristik yang berbeda:

1. **Bag of Words (BoW)**

BoW merepresentasikan dokumen sebagai vektor frekuensi kemunculan kata, tanpa memperhatikan urutan kata. Misalnya, kalimat “*the movie is good*” dan “*good movie the is*” akan memiliki representasi yang sama karena hanya menghitung jumlah kata.

2. **TF-IDF (Term Frequency–Inverse Document Frequency)**

TF-IDF tidak cuma mempertimbangkan frekuensi kemunculan kata tetapi juga mempertimbangkan seberapa jarang kata tersebut muncul di seluruh korpus. Kata yang

umum seperti *the* atau *is* akan mendapat bobot kecil, sedangkan kata yang jarang muncul seperti *excellent* atau *boring* akan mendapat bobot lebih tinggi.

3. SVD (Singular Value Decomposition)

SVD digunakan sebagai teknik reduksi dimensi pada BoW dan TF-IDF. Dengan SVD, dimensi fitur yang sangat tinggi dapat diringkas menjadi representasi berdimensi rendah (misalnya 200), sehingga lebih efisien untuk diproses. Tapi tetap ada risiko informasi penting ikut hilang.

4. Word Embedding (Word2Vec, SBERT)

- **Word2Vec**: memetakan kata ke dense vector sehingga kata-kata dengan makna serupa punya representasi yang berdekatan. Dalam eksperimen ini, representasi dokumen dihitung dengan merata-ratakan vektor kata dalam dokumen.
- **Fast Text**: memperhitungkan subword information (n-gram char). Jadi, FastText dapat menghasilkan representasi kata yang lebih baik terutama untuk kata-kata jarang muncul atau kata baru (out-of-vocabulary). Sama seperti Word2Vec, representasi dokumen diperoleh dengan merata-ratakan vektor kata.
- **GloVe (Global Vectors for Word Representation)**: GloVe dilatih secara global pada statistik co-occurrence kata dari korpus. Vektornya bersifat *pre-trained* dan sudah mengandung informasi semantik serta hubungan antar kata. Dalam eksperimen ini, dokumen direpresentasikan dengan rata-rata embedding kata dari GloVe.

Dari hasil eksperimen terlihat bahwa perbedaan performa cukup jelas antara feature extraction berbasis frekuensi (BoW/TF-IDF dengan SVD) dan feature extraction berbasis embedding (Word2Vec, FastText, GloVe).

1. Bag-of-Words (BoW) + SVD

BoW dengan reduksi dimensi (SVD) menghasilkan hasil yang cukup baik, terlihat dari skor Macro F1 yang cukup tinggi (hingga 0.736). Representasi ini cukup efektif karena dataset kecil berisi kata-kata sentimen eksplisit (*good*, *bad*, *boring*). Pola seperti ini lebih mudah ditangkap dengan pendekatan berbasis frekuensi. Namun, penggunaan SVD juga bisa mengurangi informasi.

2. TF-IDF + SVD

Hasil TF-IDF-SVD cenderung lebih rendah dibanding BoW-SVD (Macro F1 sekitar 0.628). TF-IDF lebih ditekankan pada bobot kata umum, tapi pada dataset kecil, kata umum seperti *not* atau *very* justru penting sebagai penanda sentimen sehingga TF-IDF-SVD kurang unggul.

3. Word2Vec

Word2Vec secara konsisten unggul di antara embedding lain. Macro F1 tertinggi (0.756) diperoleh dengan Word2Vec karena Word2Vec dapat menangkap kedekatan semantik antar kata, sehingga kata dengan makna serupa dipetakan berdekatan. Hal ini membantu model memahami variasi ekspresi sentimen (misalnya *excellent* $\approx$ *great*).

4. FastText

FastText punya keunggulan subword, jadi bisa memahami kata yang jarang muncul atau bentuk kata turunan. Namun, di dataset kecil, keunggulan ini tidak terlalu berguna. Skor Macro F1 (0.721) lebih rendah daripada Word2Vec. Artinya, subword generalisasi tidak terpakai optimal karena distribusi kata cukup sederhana.

5. GloVe

GloVe menunjukkan performa terendah di antara embedding (Macro F1 hanya 0.590). Kemungkinan penyebabnya adalah embedding GloVe dilatih di domain berbeda (Wikipedia, Common Crawl), sehingga tidak cocok dengan domain dataset ini.

IV. Analisis Model Deep Learning

Pada eksperimen ini diuji enam algoritma pembelajaran mesin dan deep learning untuk klasifikasi sentimen teks, dengan kombinasi berbagai teknik representasi fitur (BoW-SVD, TF-IDF-SVD, Word2Vec, FastText, GloVe).

1. Support Vector Machine (SVM)

SVM bekerja dengan mencari hyperplane yang memisahkan kelas secara optimal dengan margin terbesar. Pada data teks, terutama representasi berbasis TF-IDF atau BoW yang bersifat sparse dan berdimensi tinggi, SVM seringkali sangat efektif. Hasil eksperimen menunjukkan SVM tetap kompetitif (misalnya TFIDF-SVD + SVM dengan akurasi 0.864 dan Macro F1 0.628). Ini mengonfirmasi bahwa SVM masih menjadi baseline yang kuat untuk teks, terutama ketika fitur diekstrak melalui representasi sederhana.

2. Logistic Regression (LogReg)

LogReg memodelkan probabilitas kelas menggunakan fungsi logistik. Pada data teks, pendekatan ini sederhana namun efektif, terutama bila data tidak terlalu besar. Dari tabel, LogReg memberikan performa sedang (BoW-SVD + LogReg akurasi 0.8728, F1 0.664), masih solid tetapi cenderung kalah dari CNN. Hal ini mungkin terjadi karena LogReg hanya menangkap hubungan linear antarfitur, sehingga kurang mampu memanfaatkan representasi lebih kompleks seperti embeddings.

3. K-Nearest Neighbors (KNN)

KNN mengklasifikasikan teks berdasarkan kedekatannya dengan sampel lain di ruang fitur. Namun, kelemahannya terlihat bahwa ia memiliki performa paling rendah di antara model klasik (TFIDF-SVD + KNN Macro F1 0.55). Hal ini terjadi karena pada ruang berdimensi tinggi (ribuan kata $\rightarrow$ hasil SVD), perhitungan jarak sering tidak merepresentasikan kemiripan semantik antar teks dengan baik (*curse of dimensionality*).

4. Convolutional Neural Network (CNN)

CNN memanfaatkan filter konvolusi untuk menangkap pola lokal antar kata, misalnya *n-grams* penting yang menunjukkan sentimen. Model ini tampil paling unggul di hampir semua kombinasi embeddings, dengan skor tertinggi Word2Vec + CNN (akurasi 0.898, F1 0.756). CNN lebih efektif karena mampu mengekstrak fitur lokal dan robust terhadap variasi urutan kata, sehingga representasi semantik dari Word2Vec/FastText bisa dimanfaatkan secara maksimal.

5. Recurrent Neural Network (RNN)

RNN memproses teks secara berurutan dan menyimpan informasi melalui hidden state. Namun, hasil eksperimen menunjukkan performa RNN kurang memuaskan (misalnya Word2Vec + RNN Macro F1 hanya 0.456). Hal ini bisa dipengaruhi oleh keterbatasan dataset (kecil → rawan overfitting) serta kelemahan RNN klasik dalam menangani *long-term dependencies* dan masalah *vanishing gradient*.

6. Long Short-Term Memory (LSTM)

LSTM adalah varian RNN dengan mekanisme *gates* yang lebih baik dalam menyimpan informasi jangka panjang. Meskipun demikian, pada eksperimen ini LSTM juga gagal menunjukkan keunggulan, dengan performa setara RNN (Macro F1 = 0.456 pada semua embeddings). Ini menunjukkan bahwa untuk dataset kecil, kompleksitas LSTM justru menjadi beban, dan model cenderung *undertrained* atau overfitting meskipun dilatih lebih lama.

V. Kesimpulan

Berdasarkan hasil eksperimen, dapat disimpulkan bahwa kombinasi preprocessing yang tepat, representasi fitur sederhana maupun berbasis embedding, serta pemilihan model klasifikasi yang sesuai sangat memengaruhi performa analisis sentimen. Metode BoW-SVD dan TF-IDF-SVD cukup efektif pada dataset kecil, namun Word2Vec terbukti memberikan hasil terbaik di antara embedding dengan Macro F1 tertinggi. Dari sisi model, CNN dengan Word2Vec menghasilkan performa paling unggul (akurasi 0.898, Macro F1 0.756). KNN, RNN, dan LSTM cenderung kurang efektif karena keterbatasan data dan kompleksitas model. Secara keseluruhan, pendekatan yang paling efisien untuk analisis sentimen pada dataset dengan ukuran tidak terlalu besar adalah penggunaan CNN atau SVM dengan representasi fitur sederhana atau Word2Vec.

VI. Lampiran Hasil Tabel Perbandingan Nilai Macro F1 Score

=== FINAL RESULTS ===			
Feature	Model	Accuracy	Macro F1
Word2Vec	CNN	0.898305	0.756868
Bow-SVD	SVM	0.872881	0.735781
FastText	CNN	0.872881	0.721479
Bow-SVD	LogReg	0.872881	0.664455
TFIDF-SVD	SVM	0.864407	0.628931
Bow-SVD	KNN	0.830508	0.618123
GloVe	CNN	0.855932	0.590529
TFIDF-SVD	KNN	0.813559	0.554258
TFIDF-SVD	LogReg	0.838983	0.456221
Word2Vec	RNN	0.838983	0.456221
Word2Vec	LSTM	0.838983	0.456221
FastText	RNN	0.838983	0.456221
FastText	LSTM	0.838983	0.456221
GloVe	RNN	0.838983	0.456221
GloVe	LSTM	0.838983	0.456221